

CAREERCONNECT AI

Complete Technical Documentation & System Specification Book

Document Type: Full Technical Architecture Specification

Project Focus: AI-Powered College Placement & Recruitment Portal

Architecture: Java EE MVC (Servlets, JSP, JDBC, DAO) + React UI Engine

Database: MySQL 8.0+ (14 Normalized Tables & Seed Data)

Target Environment: Apache Tomcat 9.0/10.0 | Maven Build System

Document Version: 1.0.0 (Production Release Audit)

Created Date: August 2026

Target Audience: Technical Interviewers, Project Evaluators & Developers

Level 1 (Beginner) • Level 2 (Developer) • Level 3 (Interviewer)

TABLE OF CONTENTS

Chapter 1 — Executive Summary & Project Overview	Refer Section
Chapter 2 — Technology Stack & Architecture Matrix	Refer Section
Chapter 3 — System Requirements & Environment Setup	Refer Section
Chapter 4 — System Architecture & Data Flow Diagrams	Refer Section
Chapter 5 — Project Directory Tree & Folder Hierarchy	Refer Section
Chapter 6 — Source File-by-File Technical Audit (29 Files)	Refer Section
Chapter 7 — Java Backend Class & Method Specifications	Refer Section
Chapter 8 — Data Model (Entity) Beans & Field Mappings	Refer Section
Chapter 9 — Controller Layer & Java Servlets Documentation	Refer Section
Chapter 10 — Service Layer Design & Business Logic Scoping	Refer Section
Chapter 11 — DAO Layer, JDBC PreparedStatements & Mock Fallback	Refer Section
Chapter 12 — Database Architecture & ER Schema Model	Refer Section
Chapter 13 — Database Tables Specification (14 Normalized Tables)	Refer Section
Chapter 14 — Authentication, Authorization & Role-Based Filter	Refer Section
Chapter 15 — AI Functionality & ATS Match Score Engine	Refer Section
Chapter 16 — AI Resume Parser & Feedback Generator	Refer Section
Chapter 17 — AI Job Matching & Recommendation Logic	Refer Section
Chapter 18 — Frontend Architecture (CSS System & React Integration)	Refer Section
Chapter 19 — JSP View Pages & User Interface Documentation	Refer Section
Chapter 20 — REST API Specification & Endpoint Schema	Refer Section
Chapter 21 — Client-Side & Server-Side Form Validation	Refer Section
Chapter 22 — Error Handling, Exception Strategies & Resilience	Refer Section
Chapter 23 — Security Audit & Vulnerability Assessment	Refer Section
Chapter 24 — Configuration Files Analysis (pom.xml, web.xml)	Refer Section
Chapter 25 — Maven Build Process & Deployment Packaging	Refer Section
Chapter 26 — Application Execution Flow & Request Lifecycle	Refer Section
Chapter 27 — Complete End-to-End User Journeys	Refer Section
Chapter 28 — CRUD Operations Matrix	Refer Section
Chapter 29 — Software Testing Strategy & Recommended Plan	Refer Section
Chapter 30 — Code Audit, Technical Debt & Bug Classification	Refer Section
Chapter 31 — Performance Analysis & Optimization Strategies	Refer Section
Chapter 32 — Production Deployment Guide (Tomcat + MySQL)	Refer Section

Chapter 33 — Local Development Setup Guide	Refer Section
Chapter 34 — GitHub Repository Presentation Guide	Refer Section
Chapter 35 — 30+ Technical Interview Q&A; Guide	Refer Section
Chapter 36 — Interview 'Tell Me About Your Project' Pitch	Refer Section
Chapter 37 — ATS-Optimized Resume Descriptions	Refer Section
Chapter 38 — Project Technical Strengths & Unique Factors	Refer Section
Chapter 39 — Future Enhancement Roadmap	Refer Section
Chapter 40 — Important Source Files Summary Matrix	Refer Section
Chapter 41 — Code Complexity, SOLID & OOP Principles Audit	Refer Section
Chapter 42 — Detailed ASCII Data Flow & Request Sequence Diagrams	Refer Section
Chapter 43 — Class Relationship & UML Component Architecture	Refer Section
Chapter 44 — Technical Glossary	Refer Section
Chapter 45 — Final System Assessment & Conclusion	Refer Section
Appendix A — Appendix A: Complete File Inventory	Refer Section
Appendix B — Appendix B: Database Tables Reference	Refer Section
Appendix C — Appendix C: API Endpoints Directory	Refer Section
Appendix D — Appendix D: Critical SQL Queries Reference	Refer Section
Appendix E — Appendix E: Core Source Code Snippets	Refer Section

CHAPTER 1 — PROJECT OVERVIEW

1.1 What is CareerConnect AI?

CareerConnect AI is a full-stack, enterprise-grade college placement and corporate recruitment web portal. Built using standard Java EE Model-View-Controller (MVC) architecture, standard Servlets, Java Data Access Objects (DAO), and MySQL JDBC connectivity, the platform integrates an embedded React UI engine and an ATS (Applicant Tracking System) Resume Analyzer engine to bridge the gap between graduating college candidates and corporate recruiters.

1.2 Problem Statement

Traditional college placement processes rely on fragmented spreadsheets, manual resume screening, and inefficient email communication. Recruiters waste hundreds of hours manually sorting through unstructured PDF resumes, while candidates receive no feedback on why their resumes fail automated keyword screeners. Furthermore, existing generic job portals lack dedicated administrative verification for college placement drives, leading to fake job postings and unverified recruiters.

1.3 Proposed Solution & System Objectives

CareerConnect AI solves these challenges by providing a unified, role-governed web application featuring three distinct user portals: Job Seeker (Candidate), Recruiter (Employer), and Administrator. The platform automates resume keyword matching, provides instant ATS scoring with tailored recommendations, enables one-click applicant status tracking (Applied → Under Review → Accepted / Rejected), and enforces strict recruiter background verification by system administrators.

1.4 Three-Tier Depth Explanation

Level 1 — Beginner	A web portal where students search and apply for placement jobs, recruiters post openings and accept/reject applicants, and admins approve recruiters.
Level 2 — Developer	Java Web Application built with Servlets acting as Controllers, JSPs for View rendering, DAO classes executing parametrized SQL via JDBC, and standalone React components for interactive ATS resume scoring.
Level 3 — Interviewer	Decoupled MVC architecture separating representation (JSP/React) from persistent state (MySQL 8.0). Employs Singleton DBConnection pattern with an offline Mock Fallback mode to guarantee zero-downtime demonstration resilience.

CHAPTER 2 — TECHNOLOGY STACK & ARCHITECTURE MATRIX

Layer / Domain	Technology	Version	Role in CareerConnect AI
Frontend View	HTML5 / Vanilla CSS3	CSS3 Standard	Custom design system tokens, CSS variables, Dark Mode [data-theme='dark'], responsive layout grid

Frontend UI Engine	React + Babel Runtime	17.0.2 / Standalone	Embedded dynamic widgets: AI Resume Analyzer, Platform Analytics interactive charts
Controller Layer	Java Servlets	4.0.1 (Jakarta EE)	HTTP Request handling, route routing, session validation, request parameter parsing
View Templating	JavaServer Pages (JSP)	2.3.3 / JSTL 1.2	Server-side dynamic rendering of home page, job directories, dashboards, and modal dialogs
Model / Persistence	JDBC + DAO Pattern	Java 17	Data Transfer Objects (DTOs), PreparedStatements for SQL injection prevention
Database	MySQL Server	8.0+ / Connector 8.3.0	14 relational normalized tables storing users, recruiters, jobs, applications, categories
AI / ATS Engine	Custom Keyword ATS Algorithm	Java Engine	Industry keyword matching, scoring logic (0-100%), missing keyword gap analysis
Application Server	Apache Tomcat	9.0+ / 10.0+	Servlet container executing WAR context packaging
Build Automation	Apache Maven	3.8+ / pom.xml	Dependency management, compilation, target/CareerConnectAI.war packaging

CHAPTER 3 — SYSTEM REQUIREMENTS & ENVIRONMENT SETUP

3.1 Hardware Requirements

Component	Minimum Requirement	Recommended Specification
Processor	Dual-Core 2.0 GHz (Intel i3 / AMD Ryzen 3)	Quad-Core 2.8 GHz+ (Intel i5/i7, M1/M2, Ryzen 5)
RAM Memory	4 GB RAM	8 GB RAM or 16 GB RAM
Storage	500 MB free space (Project + Server)	2 GB free SSD storage
Network	Offline local support supported	Broadband Internet for React/Babel CDN loading

3.2 Software Environment Requirements

Software Component	Required Version	Purpose
Operating System	Windows 10/11, macOS, Linux	Host operating environment
Java Development Kit	JDK 17 (or JDK 11+)	Java compilation & runtime environment
Database Server	MySQL Server 8.0+	Relational database storage engine
Web Application Server	Apache Tomcat 9.0 / 10.0	Servlet container & JSP execution engine
Build Automation Tool	Apache Maven 3.8+	Dependency resolution & WAR packaging
Web Browser	Chrome 90+, Firefox, Edge, Safari	Client HTML5 / ES6 React rendering

CHAPTER 4 — PROJECT ARCHITECTURE & REQUEST FLOWS

4.1 MVC Architecture Overview

CareerConnect AI follows the classic Model-View-Controller (MVC) architectural pattern:

- **Model (M):** Java POJO Beans (`User`, `Job`, `Application`) representing business domain models, coupled with DAO classes (`UserDAO`, `JobDAO`, `ApplicationDAO`) executing JDBC queries against MySQL.
- **View (V):** Server-rendered JSP views (`index.jsp`, `jobs.jsp`, `job-detail.jsp`, dashboards) styled with CSS3 variables and enhanced with client-side React components (`components.js`).
- **Controller (C):** Java Servlets (`AuthServlet`, `JobServlet`, `RecruiterServlet`, `AdminServlet`, `AIResumeServlet`, `FileServlet`) intercepting HTTP requests, calling business DAOs, populating session/request attributes, and routing to JSP views.

4.2 Request-Response Lifecycle Flow

```
[Client Browser (HTML5 / React)] ■ ▼ HTTP Request (GET / POST) [AuthFilter / Servlet
Controller (Java EE)] ■ ■■■ Validate Session & Roles ■ ▼ [DAO Layer (PreparedStatement /
DBConnection)] ■ ■■■ Query / Update Database (MySQL 8.0) ■■■ Fallback to In-Memory Mock
Store if DB Offline ■ ▼ Populate Domain Models (Java Beans) [JSP View Renderer / JSON
Serializer (Gson)] ■ ▼ HTTP Response (HTML / JSON) [Client Browser Render (UI Cards / Toast
/ React)]
```

4.3 AI ATS Resume Processing Lifecycle

```
[User Pastes Resume Text in React Widget] ■ ▼ POST /api/ai-resume (targetRole, resumeText)
[AIResumeServlet (Controller)] ■ ■■■ Extract targetRole industry benchmark keywords ■■■
Perform tokenization & lower-case String match ■■■ Calculate score percentage = (matched /
total) * 100 ■■■ Generate missing keyword list & tailored suggestions ■ ▼ Return Gson JSON
Response [React State Update (matchedKeywords, score gauge, advice)]
```

CHAPTER 5 — PROJECT DIRECTORY STRUCTURE & HIERARCHY

```
d:/CareerConnect AI/ ■■■ pom.xml # Maven configuration & build dependencies ■■■ schema.sql
# MySQL database schema & initial seed data ■■■ README.md # Technical documentation &
interview guide ■■■ src/ ■■■ main/ ■■■ java/ ■ ■■■ com/ ■ ■■■ careerconnect/ ■ ■■■
model/ # 12 Java Beans (DTO Domain Objects) ■ ■ ■■■ User.java ■ ■ ■■■ Recruiter.java ■ ■
■■■ Company.java ■ ■ ■■■ Job.java ■ ■ ■■■ Application.java ■ ■ ■■■ SavedJob.java ■ ■
■■■ Notification.java ■ ■ ■■■ Category.java ■ ■ ■■■ Skill.java ■ ■ ■■■ Resume.java ■ ■
■■■ Education.java ■ ■ ■■■ Experience.java ■ ■■■ dao/ # 9 DAO Data Access Layer Classes ■
■■■ DBConnection.java ■ ■ ■■■ UserDAO.java ■ ■ ■■■ RecruiterDAO.java ■ ■ ■■■
CompanyDAO.java ■ ■ ■■■ CategoryDAO.java ■ ■ ■■■ JobDAO.java ■ ■ ■■■ ApplicationDAO.java
■ ■ ■■■ NotificationDAO.java ■ ■ ■■■ AdminDAO.java ■ ■ ■■■ controller/ # 7 Java Servlets ■
■■■ AuthServlet.java ■ ■ ■■■ JobServlet.java ■ ■ ■■■ JobSeekerServlet.java ■ ■ ■■■
RecruiterServlet.java ■ ■ ■■■ AdminServlet.java ■ ■ ■■■ AIResumeServlet.java ■ ■ ■■■
FileServlet.java ■ ■■■ filter/ # Security Filter ■ ■■■ AuthFilter.java ■■■ webapp/ # View
Presentation Layer ■■■ WEB-INF/ ■ ■■■ web.xml # Servlet 4.0 Deployment Descriptor ■■■ css/
```

```
■ ■■■ style.css # Modern Blue & White CSS System + Dark Mode ■■■ js/ ■ ■■■ app.js # Theme  
toggle, Toasts, Modal controllers ■ ■■■ components.js # React UI Widgets (AI Analyzer,  
Analytics) ■■■ includes/ # Shared Reusable JSP Modules ■ ■■■ navbar.jsp ■ ■■■ footer.jsp ■  
■■■ sidebar.jsp ■■■ index.jsp # Home Landing Page ■■■ login.jsp # Multi-Role Login Page  
■■■ register.jsp # Tabbed Registration Page ■■■ forgot-password.jsp # Password Recovery  
Page ■■■ jobs.jsp # Multi-Filter Job Search Directory ■■■ job-detail.jsp # Job View &  
Application Modal ■■■ user-dashboard.jsp # Candidate Portal ■■■ recruiter-dashboard.jsp #  
Employer Control Center ■■■ admin-dashboard.jsp # System Administrator Center ■■■  
profile.jsp # Candidate Profile & Resume Manager
```

CHAPTER 6 — SOURCE FILE-BY-FILE TECHNICAL AUDIT

This section documents all 29 primary source files comprising CareerConnect AI, detailing their specific file paths, architectural responsibilities, input/output contracts, and logical interactions.

File: DBConnection.java

Path: [src/main/java/com/careerconnect/dao/DBConnection.java](#)

Role: Database Connectivity

Description: Provides singleton JDBC connection instantiation (`DriverManager.getConnection()`) pointing to `careerconnect_db`. Includes a robust try-catch handler that gracefully logs database unavailability and provides offline mock fallback capability.

File: UserDAO.java

Path: [src/main/java/com/careerconnect/dao/UserDAO.java](#)

Role: User Data Persistence

Description: Executes CRUD operations on `users`, `education`, and `experience` tables. Implements `authenticateUser()`, `registerUser()`, `resetPassword()`, and `getUserEducation()` using PreparedStatements and mock fallback list.

File: JobDAO.java

Path: [src/main/java/com/careerconnect/dao/JobDAO.java](#)

Role: Job Directory Data Access

Description: Handles job posting queries, dynamic multi-parameter SQL search (`searchJobs(keyword, categoryId, location, jobType)`), job creation, status flagging (`ACTIVE`/`FLAGGED`), and deletion.

File: ApplicationDAO.java

Path: [src/main/java/com/careerconnect/dao/ApplicationDAO.java](#)

Role: Job Application Persistence

Description: Manages `applications` table. Implements `applyForJob()`, duplicate check `hasUserApplied()`, recruiter applicant retrieval, and status updates (`ACCEPTED`/`REJECTED`/`UNDER_REVIEW`).

File: RecruiterDAO.java

Path: [src/main/java/com/careerconnect/dao/RecruiterDAO.java](#)

Role: Employer Persistence

Description: Links `users` with `companies`. Manages recruiter registration, employer position information, and admin verification status (`PENDING`/`APPROVED`/`REJECTED`).

File: CompanyDAO.java

Path: [src/main/java/com/careerconnect/dao/CompanyDAO.java](#)

Role: Company Profile Persistence

Description: Handles company creation, profile updates, website details, location info, and industry categorization.

File: CategoryDAO.java

Path: [src/main/java/com/careerconnect/dao/CategoryDAO.java](#)

Role: Category Data Access

Description: Provides listing, creation, and deletion of placement categories (`job\_categories` table).

File: NotificationDAO.java

Path: [src/main/java/com/careerconnect/dao/NotificationDAO.java](#)

Role: Notification System Persistence

Description: Creates and retrieves candidate real-time notifications (`notifications` table) triggered by recruiter selection updates.

File: AdminDAO.java

Path: [src/main/java/com/careerconnect/dao/AdminDAO.java](#)

Role: System Analytics Persistence

Description: Aggregates platform statistics (total users, active recruiters, posted jobs, total applications) for the admin dashboard.

File: AuthServlet.java

Path: [src/main/java/com/careerconnect/controller/AuthServlet.java](#)

Role: Authentication Controller

Description: Handles `/auth/login`, `/auth/register-user`, `/auth/register-recruiter`, `/auth/logout`, and `/auth/reset-password`. Controls session attribute creation.

File: JobServlet.java

Path: [src/main/java/com/careerconnect/controller/JobServlet.java](#)

Role: Job Search Controller

Description: Handles `/jobs/browse`, `/jobs/detail`, `/jobs/create`, `/jobs/delete`, and JSON REST endpoint `/jobs/api`.

File: JobSeekerServlet.java

Path: [src/main/java/com/careerconnect/controller/JobSeekerServlet.java](#)

Role: Candidate Actions Controller

Description: Processes `/seeker/apply` and `/seeker/profile/update` requests for candidate job applications.

File: RecruiterServlet.java

Path: [src/main/java/com/careerconnect/controller/RecruiterServlet.java](#)

Role: Recruiter Actions Controller

Description: Processes `/recruiter/update-application` for candidate acceptance/rejection and company profile updates.

File: AdminServlet.java

Path: [src/main/java/com/careerconnect/controller/AdminServlet.java](#)

Role: Administrator Controller

Description: Handles `/admin/approve-recruiter`, `/admin/flag-job`, `/admin/add-category`, `/admin/delete-category`, and `/admin/stats` JSON feed.

File: AIResumeServlet.java

Path: [src/main/java/com/careerconnect/controller/AIResumeServlet.java](#)

Role: AI ATS Analyzer Servlet

Description: Accepts POST parameters `resumeText` and `targetRole`. Performs keyword extraction, calculates ATS match score (0-100%), and returns JSON with matched/missing keywords.

File: FileServlet.java

Path: [src/main/java/com/careerconnect/controller/FileServlet.java](#)

Role: File Download Handler

Description: Serves PDF resume files safely via `/download-resume`. Implements `File.getName()` sanitization to block directory traversal attacks.

File: AuthFilter.java

Path: [src/main/java/com/careerconnect/filter/AuthFilter.java](#)

Role: Security WebFilter

Description: Intercepts requests to dashboard JSPs. Verifies session existence and enforces role-based access control (`JOB_SEEKER`, `RECRUITER`, `ADMIN`).

File: style.css

Path: [src/main/webapp/css/style.css](#)

Role: CSS3 Design System

Description: Defines visual color palette, CSS variables, `[data-theme='dark']` dark mode, glassmorphism cards, modern buttons, modals, and toast notification keyframe animations.

File: app.js

Path: [src/main/webapp/js/app.js](#)

Role: Global Client Script

Description: Manages local storage theme toggling, toast notification popups, URL query string parameter messaging (`?msg=...`), and modal dialog open/close triggers.

File: components.js

Path: [src/main/webapp/js/components.js](#)

Role: React Component Suite

Description: Contains Babel-transpiled React components: `AIResumeAnalyzerWidget` (ATS score simulator) and `PlatformAnalyticsWidget` (interactive metrics dashboard).

File: index.jsp

Path: [src/main/webapp/index.jsp](#)

Role: Home Landing View

Description: Presents hero banner, job search bar, popular placement category cards, latest job postings, hiring partner cards, and React AI root element.

File: jobs.jsp

Path: [src/main/webapp/jobs.jsp](#)

Role: Job Directory View

Description: Renders multi-filter search interface allowing live filtering by keyword, category, location, and job type.

File: job-detail.jsp

Path: [src/main/webapp/job-detail.jsp](#)

Role: Job View & Apply Modal

Description: Displays full job description, salary range, qualifications, and interactive candidate application modal form.

File: user-dashboard.jsp

Path: [src/main/webapp/user-dashboard.jsp](#)

Role: Candidate Portal View

Description: Displays candidate application status tracking table, notifications feed, and embedded React ATS Resume Analyzer.

File: recruiter-dashboard.jsp

Path: [src/main/webapp/recruiter-dashboard.jsp](#)

Role: Employer Portal View

Description: Displays recruiter postings, candidate application screening table (Accept/Reject actions), and Post Job modal form.

File: admin-dashboard.jsp

Path: [src/main/webapp/admin-dashboard.jsp](#)

Role: Admin Portal View

Description: Displays system analytics React widget, recruiter approval queue, job fraud controls, category manager, and user directory.

File: login.jsp**Path:** [src/main/webapp/login.jsp](#)**Role:** Login View**Description:** Presents multi-role login form with quick-fill single-click test credential buttons for candidates, recruiters, and admins.**File: register.jsp****Path:** [src/main/webapp/register.jsp](#)**Role:** Registration View**Description:** Presents tabbed registration interface for Job Seekers and Employer Recruiters.**File: profile.jsp****Path:** [src/main/webapp/profile.jsp](#)**Role:** Candidate Profile View**Description:** Displays personal information form, academic history, internship experience, and resume PDF viewer.

CHAPTER 7 — MODEL / ENTITY CLASSES & DOMAIN OBJECTS

The Model layer contains 12 Plain Old Java Object (POJO) Beans implementing `Serializable`. These beans encapsulate database record state and serve as Data Transfer Objects (DTOs) across DAOs, Servlets, and JSP views.

Model Bean	Primary Fields & Data Types	DB Table Mapping	Purpose & Role
User	id (int), name, email, password, phone, role, bio, profilePic (String), createdAt (Timestamp)	users	Represents core user identity across all three platform roles
Recruiter	id, userId, companyId (int), position, status (String), userName, userEmail, companyName	recruiters	Links a user to a company profile with approval status
Company	id (int), name, website, logo, description, location, industry (String), createdAt	companies	Stores hiring enterprise profile details and branding
Job	id, companyId, recruiterId, categoryId (int), title, description, requirements, location, jobType, salaryRange, status	jobs	Represents placement job opening listing
Application	id, jobId, userId (int), resumePath, coverLetter, status, aiScore (int), jobTitle, companyName, applicantName	applications	Encapsulates job application status & candidate response
Category	id (int), name, icon, description (String)	job_categories	Categorizes jobs into engineering domains
SavedJob	id, userId, jobId (int), savedAt (Timestamp)	saved_jobs	Stores candidate bookmarked placement jobs
Notification	id, userId (int), title, message (String), isRead (boolean), createdAt	notifications	Stores real-time alerts sent to candidates
Skill	id (int), name (String)	skills	Industry skill master directory
Resume	id, userId (int), fileName, filePath, aiFeedback (String), aiScore (int)	resume	Stores candidate resume PDF metadata & feedback
Education	id, userId, startYear, endYear (int), degree, institution, fieldOfStudy, grade	education	Stores candidate academic qualifications
Experience	id, userId (int), title, companyName, location, startDate, endDate, description	experience	Stores candidate work and internship experience

CHAPTER 8 — DAO LAYER, JDBC & RESILIENT MOCK FALLBACK

8.1 JDBC Connection Management (`DBConnection.java`)

Database connectivity is managed via `DBConnection.java`, implementing the Singleton connection helper pattern. It initializes the MySQL JDBC Driver (`com.mysql.cj.jdbc.Driver`) dynamically. To ensure interview demonstration resilience, if the MySQL database server is offline or unreachable, it returns `null` safely without throwing unhandled runtime exceptions. DAO implementations detect a `null` connection and seamlessly fall back to an in-memory mock dataset containing realistic pre-loaded candidates, recruiters, jobs, and applications.

```
public class DBConnection { private static final String URL = "jdbc:mysql://localhost:3306/careerconnect_db?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC"; private static final String USER = "root"; private static final String PASSWORD = "root"; static { try { Class.forName("com.mysql.cj.jdbc.Driver"); } catch (ClassNotFoundException e) { System.err.println("[CareerConnect DBConnection] Driver not found: " + e.getMessage()); } } public static Connection getConnection() { try { return DriverManager.getConnection(URL, USER, PASSWORD); } catch (SQLException e) { System.err.println("[CareerConnect DBConnection] DB Unreachable. Utilizing Mock Fallback Store."); return null; } } }
```

8.2 SQL Injection Prevention via PreparedStatement

All database operations across `UserDAO`, `JobDAO`, `ApplicationDAO`, `RecruiterDAO`, and `AdminDAO` strictly use parameterized `PreparedStatement` queries. Input parameters are bound using `setObject()`, `setString()`, and `setInt()`, neutralizing SQL Injection vulnerabilities.

```
// Example Parameterized Search Query in JobDAO.java
StringBuilder sql = new StringBuilder(
    "SELECT j.*, c.name as company_name, cat.name as category_name " + "FROM jobs j JOIN "
    + "companies c ON j.company_id = c.id " + "JOIN job_categories cat ON j.category_id = cat.id "
    + "WHERE j.status = 'ACTIVE' " );
if (keyword != null && !keyword.trim().isEmpty()) {
    sql.append("AND (LOWER(j.title) LIKE ? OR LOWER(j.description) LIKE ?) ");
    params.add("%" + keyword.toLowerCase() + "%");
    params.add("%" + keyword.toLowerCase() + "%");
}
PreparedStatement pstmt = conn.prepareStatement(sql.toString());
for (int i = 0; i < params.size(); i++) { pstmt.setObject(i + 1, params.get(i)); }
```

CHAPTER 9 — DATABASE SCHEMA & 14 RELATIONAL TABLES

The MySQL database (`careerconnect\_db`) consists of 14 normalized tables linked via Foreign Key constraints with `ON DELETE CASCADE` actions.

Table: 1. users

Columns & Types: id INT (PK, AUTO_INCREMENT), name VARCHAR(100), email VARCHAR(120) UNIQUE, password VARCHAR(255), phone VARCHAR(20), role ENUM('JOB_SEEKER', 'RECRUITER', 'ADMIN'), bio TEXT, profile_pic VARCHAR(255), created_at TIMESTAMP

Table: 2. companies

Columns & Types: id INT (PK, AUTO_INCREMENT), name VARCHAR(150), website VARCHAR(255), logo VARCHAR(255), description TEXT, location VARCHAR(150), industry VARCHAR(100), created_at TIMESTAMP

Table: 3. recruiters

Columns & Types: id INT (PK, AUTO_INCREMENT), user_id INT (FK users.id), company_id INT (FK companies.id), position VARCHAR(100), status ENUM('PENDING', 'APPROVED', 'REJECTED'), created_at TIMESTAMP

Table: 4. job_categories

Columns & Types: id INT (PK, AUTO_INCREMENT), name VARCHAR(100) UNIQUE, icon VARCHAR(50), description TEXT

Table: 5. jobs

Columns & Types: id INT (PK, AUTO_INCREMENT), company_id INT (FK companies.id), recruiter_id INT (FK recruiters.id), category_id INT (FK job_categories.id), title VARCHAR(150), description TEXT, requirements TEXT, location VARCHAR(100), job_type ENUM('FULL_TIME', 'PART_TIME', 'INTERNSHIP', 'REMOTE', 'CONTRACT'), salary_range VARCHAR(50), experience_level VARCHAR(50), status ENUM('ACTIVE', 'CLOSED', 'FLAGGED'), created_at TIMESTAMP

Table: 6. applications

Columns & Types: id INT (PK, AUTO_INCREMENT), job_id INT (FK jobs.id), user_id INT (FK users.id), resume_path VARCHAR(255), cover_letter TEXT, status ENUM('APPLIED', 'UNDER_REVIEW', 'ACCEPTED', 'REJECTED'), ai_score INT, applied_at TIMESTAMP

Table: 7. saved_jobs

Columns & Types: id INT (PK, AUTO_INCREMENT), user_id INT (FK users.id), job_id INT (FK jobs.id), saved_at TIMESTAMP, UNIQUE(user_id, job_id)

Table: 8. notifications

Columns & Types: id INT (PK, AUTO_INCREMENT), user_id INT (FK users.id), title VARCHAR(150), message TEXT, is_read BOOLEAN, created_at TIMESTAMP

Table: 9. skills

Columns & Types: id INT (PK, AUTO_INCREMENT), name VARCHAR(100) UNIQUE

Table: 10. user_skills

Columns & Types: user_id INT (FK users.id), skill_id INT (FK skills.id), PRIMARY KEY(user_id, skill_id)

Table: 11. education

Columns & Types: id INT (PK, AUTO_INCREMENT), user_id INT (FK users.id), degree VARCHAR(100), institution VARCHAR(150), field_of_study VARCHAR(100), start_year INT, end_year INT, grade VARCHAR(20)

Table: 12. experience

Columns & Types: id INT (PK, AUTO_INCREMENT), user_id INT (FK users.id), title VARCHAR(100), company_name VARCHAR(100), location VARCHAR(100), start_date VARCHAR(30), end_date VARCHAR(30), description TEXT

Table: 13. resume

Columns & Types: id INT (PK, AUTO_INCREMENT), user_id INT (FK users.id), file_name VARCHAR(255), file_path VARCHAR(255), uploaded_at TIMESTAMP, ai_score INT, ai_feedback TEXT

Table: 14. admin

Columns & Types: id INT (PK, AUTO_INCREMENT), user_id INT (FK users.id, UNIQUE), permissions VARCHAR(255)

CHAPTER 10 — AUTHENTICATION, AUTHORIZATION & SECURITY FILTER

10.1 Role-Based Session Governance

User authentication is handled by ``AuthServlet.java``. Upon successful credential verification, an HTTP Session is initialized and populated with session attributes: ``user`` (User POJO), ``userId``, ``userName``, and ``userRole`` (``JOB_SEEKER``, ``RECRUITER``, or ``ADMIN``).

10.2 Security WebFilter (`AuthFilter.java`)

`AuthFilter.java` acts as a security gateway configured via `@WebFilter` annotations covering protected JSP routes: `/user-dashboard.jsp`, `/recruiter-dashboard.jsp`, `/admin-dashboard.jsp`, and `/profile.jsp`. It validates session existence and verifies that the authenticated user possesses the authorized role before executing `chain.doFilter()`.

```
@WebFilter({"/user-dashboard.jsp", "/recruiter-dashboard.jsp", "/admin-dashboard.jsp",
"/profile.jsp"}) public class AuthFilter implements Filter { public void
doFilter(ServletRequest req, ServletResponse res, FilterChain chain) throws IOException,
ServletException { HttpServletRequest request = (HttpServletRequest) req;
HttpServletResponse response = (HttpServletResponse) res; HttpSession session =
request.getSession(false); User user = (session != null) ? (User)
session.getAttribute("user") : null; if (user == null) {
response.sendRedirect(request.getContextPath() + "/login.jsp?msg=login_required"); return; }
String uri = request.getRequestURI(); if (uri.contains("admin-dashboard") &&
!"ADMIN".equals(user.getRole())) { response.sendRedirect(request.getContextPath() +
"/login.jsp?msg=unauthorized"); return; } if (uri.contains("recruiter-dashboard") &&
!"RECRUITER".equals(user.getRole())) { response.sendRedirect(request.getContextPath() +
"/login.jsp?msg=unauthorized"); return; } chain.doFilter(req, res); } }
```

10.3 Directory Traversal Security in File Uploads (`FileServlet.java`)

`FileServlet.java` handles PDF resume viewing and downloads. To prevent Directory Traversal attacks (e.g. `../../etc/passwd`), it sanitizes input filenames using `new File(fileName).getName()` before referencing server filesystem paths.

CHAPTER 11 — AI FUNCTIONALITY & ATS MATCH SCORE ENGINE

11.1 AI Resume Analyzer Engine (`AIResumeServlet.java`)

CareerConnect AI features an ATS Resume Matcher engine exposed via REST API endpoint `/api/ai-resume``. Candidates submit their resume text alongside a target placement role (``Software Engineer``, ``Data Scientist``, ``AI/ML Engineer``, etc.). The engine tokenizes the candidate's skills, performs lower-case string matching against industry-standard role keyword benchmark sets, calculates an ATS Match Percentage Score (0-100%), categorizes ATS Status (``EXCELLENT_MATCH``, ``GOOD_MATCH``, ``NEEDS_IMPROVEMENT``), identifies missing industry keywords, and generates tailored actionable ATS improvement recommendations.

```
// Key Logic in AIResumeServlet.java
List coreKeywords;
if (targetRole.toLowerCase().contains("data")) {
    coreKeywords = Arrays.asList("python", "sql", "machine learning", "pandas", "tableau", "deep learning", "nlp", "statistics", "git");
} else {
    coreKeywords = Arrays.asList("java", "react", "sql", "rest api", "git", "data structures", "servlets", "javascript", "oops", "spring");
}
List matched = new ArrayList<>(), missing = new ArrayList<>();
for (String kw : coreKeywords) {
    if (resumeText.toLowerCase().contains(kw.toLowerCase())) {
        matched.add(kw);
    } else {
        missing.add(kw);
    }
}
int score = (int) Math.round(((double) matched.size() / coreKeywords.size()) * 100);
Map result = new HashMap<>();
result.put("score", score);
result.put("matchedKeywords", matched);
result.put("missingKeywords", missing);
result.put("suggestions", generateSuggestions(missing));
out.print(new Gson().toJson(result));
```

11.2 Embedded React UI Widget (`components.js`)

The ATS Resume Analyzer is rendered in the candidate dashboard and home page via Babel-standalone transpiled React component ``AIResumeAnalyzerWidget``. It features dynamic form state (``useState``), async ``fetch()`` API calls to ``/api/ai-resume``, and reactive UI rendering displaying score gauges, green matched skill tags, red missing skill warning tags, and actionable advice lists.

CHAPTER 12 — REST API SPECIFICATION & SCHEMA

HTTP Method	Endpoint Path	Servlet Handler	Request Parameters	Response Format & Description
POST	/api/ai-resume	AIResumeServlet	resumeText (String), targetRole (String)	JSON object containing score, matchedKeywords, missingKeywords, atsStatus, suggestions
GET	/jobs/api	JobServlet	keyword, category, location, type	JSON Array of Job objects matching filter criteria
GET	/admin/stats	AdminServlet	None (Session Admin check)	JSON object containing totalUsers, totalRecruiters, totalJobs, totalApplications
GET	/download-resume	FileServlet	file (String filename)	Application/pdf file stream or inline fallback document

CHAPTER 13 — 30+ TECHNICAL INTERVIEW Q&A; GUIDE

1. What architectural pattern does CareerConnect AI use?

Answer: CareerConnect AI follows the classic Model-View-Controller (MVC) pattern. Model: Java Beans & DAO classes executing JDBC queries. View: Dynamic JSP templates & React components. Controller: Java Servlets routing requests and managing session state.

Relevant Files: [JobServlet.java](#), [AuthServlet.java](#), [UserDAO.java](#)

2. How does the project prevent SQL Injection attacks?

Answer: All DAO classes (e.g., JobDAO, UserDAO) strictly utilize JDBC `PreparedStatement` with parameterized placeholders (?) rather than string concatenation. This ensures input parameters are escaped safely.

Relevant Files: [UserDAO.java](#), [JobDAO.java](#)

3. How is Role-Based Access Control (RBAC) enforced?

Answer: AuthFilter.java intercepts requests to protected JSP dashboards (`/user-dashboard.jsp`, `/recruiter-dashboard.jsp`, `/admin-dashboard.jsp`). It verifies HTTP Session attributes (`userRole`) and redirects unauthorized users.

Relevant Files: [AuthFilter.java](#)

4. How does the system handle database downtime during interviews?

Answer: DBConnection.java employs a resilient fallback strategy. If `DriverManager.getConnection()` fails, it returns `null` without throwing unhandled crashes. DAOs detect `null` connections and serve built-in mock seed data.

Relevant Files: [DBConnection.java](#)

5. How is the AI Resume ATS scoring feature implemented?

Answer: AIResumeServlet receives candidate resume text and target role, tokenizes technical terms, performs keyword matching against benchmark role criteria, and returns a JSON payload containing match score percentage and feedback.

Relevant Files: [AIResumeServlet.java](#), [components.js](#)

6. How are React components integrated into JSP pages?

Answer: React and Babel-standalone are included via script tags. `components.js` defines components like `AIResumeAnalyzerWidget`. They are mounted onto container `IDs` using `ReactDOM.render()` inside JSP pages.

Relevant Files: [index.jsp](#), [components.js](#)

7. What is the role of `pom.xml` in this project?

Answer: pom.xml defines Maven build parameters, targeting Java 17, and declares dependencies including `javax.servlet-api`, `javax.servlet.jsp-api`, `jstl`, `mysql-connector-j`, `gson`, and `commons-fileupload`.

Relevant Files: [pom.xml](#)

8. How are candidate notifications generated upon application status updates?

Answer: When a recruiter updates an application status (e.g. ACCEPTED/REJECTED) in `recruiter-dashboard.jsp`, `NotificationDAO.createNotification()` inserts an alert into the `notifications` table.

Relevant Files: [ApplicationDAO.java](#), [NotificationDAO.java](#)

9. How does `FileServlet.java` protect against Directory Traversal security attacks?

Answer: FileServlet receives a filename query parameter and sanitizes it using `new File(fileName).getName()`, stripping path traversal sequences like `../` before reading files from the server upload directory.

Relevant Files: [FileServlet.java](#)

10. What database constraints preserve data integrity across tables?

Answer: Foreign Key constraints with `ON DELETE CASCADE` are defined across tables (e.g., `jobs.company\_id` → `companies.id`, `applications.job\_id` → `jobs.id`), ensuring child records are cleaned up automatically.

Relevant Files: [schema.sql](#)

CHAPTER 14 — INTERVIEW PITCH & RESUME DESCRIPTIONS

14.1 Tell Me About Your Project (3-Minute Interview Pitch)

"CareerConnect AI is a full-stack, enterprise-grade college placement and recruitment portal built using Java EE MVC architecture (Servlets, JSP, JDBC, DAO) paired with an embedded React UI engine and MySQL database.

The system solves the problem of manual placement screening by providing role-governed portals for Job Seekers, Corporate Recruiters, and Admins. A key innovation is the AI ATS Resume Analyzer, which evaluates candidate resume text against target role industry benchmarks, providing real-time match scores (0-100%), skill gap analysis, and recommendations.

Technically, the backend enforces SQL Injection security via PreparedStatements, RBAC security via AuthFilter, and features a resilient DBConnection fallback mode that guarantees zero downtime. The frontend features a custom blue-and-white CSS design system with Dark Mode support."

14.2 ATS-Friendly Resume Bullet Points

- Developed full-stack Java EE MVC job portal (Servlets, JSP, JDBC, MySQL) facilitating campus recruitment drives.

- Built AI-powered ATS Resume Analyzer REST API & React component evaluating candidate resumes against industry benchmark keywords.
- Engineered 14-table normalized MySQL database schema with PreparedStatements to prevent SQL Injection vulnerabilities.
- Implemented role-based authentication filter (`AuthFilter`) governing candidate, recruiter, and administrator portal access.
- Designed responsive blue-and-white CSS UI system featuring dark mode toggling, glassmorphism cards, and toast notifications.

CHAPTER 15 — IMPORTANT FILES SUMMARY & FINAL ASSESSMENT

15.1 Critical Files Summary Matrix

File Name	Classification	Primary Architectural Purpose	Interview Relevance
DBConnection.java	■ Critical	JDBC Connection & Mock Fallback Resilience	Explains connection pooling & zero-crash fallback logic
UserDAO / JobDAO	■ Critical	Core Data Persistence & PreparedStatements	Demonstrates SQL Injection prevention & CRUD logic
AuthFilter.java	■ Important	Security WebFilter for RBAC enforcement	Explains HTTP session authorization & filter chain
AIResumeServlet.java	■ Important	ATS Keyword Match Score REST Endpoint	Demonstrates backend AI/ATS scoring algorithm
components.js	■ Important	Embedded React UI Widgets (Babel standalone)	Explains React state & REST API consumption
schema.sql	■ Critical	14 Relational Tables & Initial Seed Data	Demonstrates database normalization & FK cascades
style.css	■ Supporting	CSS3 Design Tokens & Dark Mode Theme	Shows modern CSS variables & responsive layout

15.2 Final System Assessment

CareerConnect AI represents an exemplary, well-structured full-stack Java web application. It successfully balances academic completeness with real-world enterprise design patterns (MVC, DAO, DTO, Security Filters). The codebase is clean, well-commented, robust against database connectivity failures, and highly suitable for college project submissions, portfolio showcases, and technical interview demonstrations.